

# Datumové a časové funkce v PostgreSQL na databázi Sakila

---

Příklady používají schema `public` a plně kvalifikované názvy tabulek.

Všechny příklady jsou pouze dotazy `SELECT` a nemění data.

## 1. Zobrazení data a času

Sloupce `rental_date`, `return_date`, `payment_date` a `create_date` mají typ `timestamp without time zone`. Pomocí `CAST` nebo operátoru `::date` lze zobrazit pouze datum.

```
SELECT
    rental_id,
    rental_date,
    rental_date::date AS rental_day,
    rental_date::time AS rental_time,
    return_date,
    return_date::date AS return_day
FROM public.rental
ORDER BY rental_date
LIMIT 20;
```

`::date` je kratší PostgreSQL zápis pro převod na typ `date`. Přebod na `time` ponechá pouze časovou část.

## 2. Získání jednotlivých částí data pomocí `EXTRACT`

`EXTRACT` získá z data nebo času konkrétní část, například rok, měsíc, den nebo hodinu.

```
SELECT
    rental_id,
    rental_date,
    EXTRACT(YEAR FROM rental_date) AS rental_year,
    EXTRACT(MONTH FROM rental_date) AS rental_month,
    EXTRACT(DAY FROM rental_date) AS rental_day,
    EXTRACT(HOUR FROM rental_date) AS rental_hour
FROM public.rental
ORDER BY rental_date
LIMIT 20;
```

Výsledkem `EXTRACT` je číselná hodnota, se kterou lze dále počítat nebo ji použít v podmínce.

## 3. Formátování data pomocí `TO_CHAR`

`TO_CHAR` převede datum nebo čas na text podle formátovací masky.

```

SELECT
    payment_id,
    payment_date,
    TO_CHAR(payment_date, 'DD.MM.YYYY') AS payment_date_czech,
    TO_CHAR(payment_date, 'YYYY-MM') AS payment_month,
    TO_CHAR(payment_date, 'TMMonth YYYY') AS payment_month_name
FROM public.payment
ORDER BY payment_date
LIMIT 20;

```

Příklady formátovacích značek:

- **DD** znamená den;
- **MM** znamená měsíc jako číslo;
- **YYYY** znamená čtyřmístný rok;
- **Month** znamená název měsíce.

**TO\_CHAR** je vhodný pro výstup pro uživatele, ale výsledek je text. Pro filtrování a řazení je obvykle lepší ponechat původní datový typ.

## 4. Začátek období pomocí **DATE\_TRUNC**

**DATE\_TRUNC** zaokrouhlí datum na začátek zvoleného období, například roku, měsíce nebo dne.

```

SELECT
    payment_id,
    payment_date,
    DATE_TRUNC('month', payment_date) AS first_day_of_month,
    DATE_TRUNC('year', payment_date) AS first_day_of_year
FROM public.payment
ORDER BY payment_date
LIMIT 20;

```

Funkce je užitečná pro převedení různých časových okamžiků na společný začátek období, například na první den měsíce.

## 5. Výpočet délky výpůjčky

Odečtením dvou hodnot typu **timestamp** vznikne interval. Pomocí **AGE** lze také získat rozdíl mezi dvěma daty v kalendářním tvaru.

```

SELECT
    rental_id,
    rental_date,
    return_date,
    return_date - rental_date AS rental_duration,
    AGE(return_date, rental_date) AS rental_age
FROM public.rental

```

```
WHERE return_date IS NOT NULL
ORDER BY rental_duration DESC
LIMIT 20;
```

Podmínka `return_date IS NOT NULL` vyloučí výpůjčky, které ještě nebyly vráceny. Bez této podmínky by výpočet délky vrátil `NULL`.

## 6. Denní složka intervalu pomocí `DATE_PART`

`DATE_PART('day', interval)` vrací denní složku intervalu. Odečtení dvou hodnot `timestamp` vytváří interval bez měsíční složky, takže v následujícím dotazu jde o počet celých dní výpůjčky. U intervalu obsahujícího měsíce, například výsledku `AGE`, samotná denní složka nepředstavuje celkovou délku ve dnech.

```
SELECT
    rental_id,
    rental_date,
    return_date,
    DATE_PART('day', return_date - rental_date) AS rental_days
FROM public.rental
WHERE return_date IS NOT NULL
ORDER BY rental_days DESC, rental_id
LIMIT 20;
```

U běžných výpůjček je možné použít také přímé odečtení hodnot typu `date`, jehož výsledkem je počet dní.

```
SELECT
    rental_id,
    return_date::date - rental_date::date AS rental_days
FROM public.rental
WHERE return_date IS NOT NULL
ORDER BY rental_days DESC, rental_id
LIMIT 20;
```

## 7. Aktuální datum a čas

PostgreSQL nabízí několik funkcí pro aktuální datum a čas.

```
SELECT
    CURRENT_DATE AS current_date,
    CURRENT_TIME AS current_time,
    CURRENT_TIMESTAMP AS current_timestamp,
    NOW() AS now_value;
```

`CURRENT_DATE` vrací pouze datum. `CURRENT_TIMESTAMP` a `NOW()` vracejí datum i čas. Tyto hodnoty vycházejí ze začátku aktuální transakce, a proto se během dlouhé transakce při dalších dotazech nemění.

## 8. Filtr podle období

Při výběru záznamů podle sloupce typu `timestamp` je důležité uvědomit si, že datum bez uvedeného času znamená půlnoc, tedy `00:00:00`. Podmínka `payment_date <= DATE '2005-05-31'` proto zahrne pouze okamžik na začátku 31. května, nikoli celý tento den.

```
SELECT
    payment_id,
    customer_id,
    amount,
    payment_date
FROM public.payment
WHERE payment_date >= DATE_TRUNC('month', payment_date)
ORDER BY payment_date;
```

Uvedená podmínka je vždy pravdivá pro neprázdné datum a slouží hlavně jako ukázka práce s `DATE_TRUNC`. Pro praktické filtrování konkrétního období je bezpečné použít dolní hranici včetně (`>=`) a horní hranici bez zahrnutí (`<`):

```
SELECT
    payment_id,
    customer_id,
    amount,
    payment_date
FROM public.payment
WHERE payment_date >= TIMESTAMP '2005-05-01 00:00:00'
    AND payment_date < TIMESTAMP '2005-06-01 00:00:00'
ORDER BY payment_date;
```

Horní hranice je záměrně nastavena jako `<`, nikoli `<=`. Díky tomu dotaz zahrne celý květen, včetně všech časů a zlomků sekundy 31. května, ale nezahrne první okamžik června.

## 9. Konverze mezi `date` a `timestamp`

Typ `date` obsahuje pouze kalendářní datum. Typ `timestamp` obsahuje datum i čas. Při převodu z `date` na `timestamp` PostgreSQL doplní čas `00:00:00`; při převodu z `timestamp` na `date` časová část odpadne.

```
SELECT
    rental_id,
    rental_date,
    CAST(rental_date AS date) AS rental_date_only,
    CAST(rental_date AS timestamp) AS rental_timestamp,
    rental_date::date AS rental_date_short_cast,
    rental_date::timestamp AS rental_timestamp_short_cast
FROM public.rental
ORDER BY rental_date
LIMIT 20;
```

Explicitní zápis pomocí `CAST` je čitelný a přenositelný. PostgreSQL zápis `::date` nebo `::timestamp` je kratší. Při převodu `timestamp` na `date` se čas nezaokrouhluje, ale jednoduše se odstraní.

## 10. Kontrola nevrácených výpůjček

Kombinací `CURRENT_TIMESTAMP` a `CASE` lze označit výpůjčky jako vrácené nebo stále aktivní.

```
SELECT
    rental_id,
    rental_date,
    return_date,
    CASE
        WHEN return_date IS NULL THEN 'nevráceno'
        WHEN return_date <= CURRENT_TIMESTAMP THEN 'vráceno'
        ELSE 'datum vrácení je v budoucnosti'
    END AS rental_status
FROM public.rental
ORDER BY rental_date;
```

`CASE` nejprve ošetří `NULL` hodnotu. Teprve u vrácených výpůjček se porovnává datum vrácení s aktuálním časem.

## Přehled nejčastějších funkcí

| Funkce nebo operátor                  | Účel                                   |
|---------------------------------------|----------------------------------------|
| <code>CURRENT_DATE</code>             | aktuální datum                         |
| <code>CURRENT_TIMESTAMP, NOW()</code> | aktuální datum a čas                   |
| <code>EXTRACT</code>                  | získání roku, měsíce, dne nebo hodiny  |
| <code>DATE_PART</code>                | získání části data nebo intervalu      |
| <code>DATE_TRUNC</code>               | zaokrouhlení na začátek období         |
| <code>TO_CHAR</code>                  | formátování data jako textu            |
| <code>AGE</code>                      | rozdíl mezi dvěma daty jako interval   |
| <code>date2 - date1</code>            | rozdíl mezi daty nebo timestampy       |
| <code>INTERVAL</code>                 | časový interval pro výpočty            |
| <code>::date, CAST</code>             | převod datového typu                   |
| <code>IS NULL, COALESCE</code>        | práce s chybějícím datem               |
| <code>CASE</code>                     | vytvoření slovního označení podle data |

Při tvorbě tohoto dokumentu byl využit GitHub Copilot.